

Mecanismos de Sincronização com OpenMP

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Esta tarefa implementa o estimador de π por Monte Carlo com N pontos uniformes em $[-1, 1]^2$ e gerador `rand_r` com semente determinística por iteração. Cinco versões são comparadas: (v1) contador compartilhado com `#pragma omp critical` por hit, (v2) contador compartilhado com `#pragma omp atomic` por hit, (v3) contador privado por thread com um único `critical` final, (v4) vetor de parciais por thread com soma serial final e (v5) cláusula `reduction`. Comparam-se desempenho e produtividade de cada mecanismo e apresenta-se um roteiro direto de quando utilizar cada um, incluindo `critical` nomeadas e locks explícitos.

2 Fundamentação teórica

2.1 Monte Carlo e semente por iteração

O estimador utiliza

$$\hat{\pi} = 4 \cdot \frac{n_{dentro}}{N},$$

com pontos uniformemente distribuídos em $[-1, 1]^2$.

Para isolar o custo de sincronização, cada iteração i gera uma semente determinística própria:

```
seed = 12345u + i*2654435761u
```

Esse procedimento, adotado nesta tarefa, emprega o hash de Knuth e extrai x e y por meio de duas chamadas a `rand_r(&seed)`.

Assim, o mesmo valor de N produz os mesmos pontos e o mesmo valor de `hits` com 1 ou 28 threads e em todas as versões (v1 a v5), sem misturar o efeito do gerador de números aleatórios com o efeito da sincronização.

2.2 `critical`, `atomic` e `reduction`

`critical` serializa um bloco estruturado qualquer utilizando um mecanismo interno de exclusão mútua. O mecanismo pode proteger qualquer conjunto de operações, mas cada entrada na região crítica possui um custo de sincronização.

`atomic` cobre uma única operação escalar de leitura-modificação-escrita, por exemplo, `hits++`, utilizando uma operação atômica de hardware. Apresenta custo menor que `critical` em operações simples, mas sua utilização é restrita a um endereço e a uma operação compatível. O restante de uma expressão, como `f()`, é executado fora da região atômica.

`reduction(+ hits)` fornece a cada thread um acumulador privado e realiza a combinação dos resultados ao final da região paralela. O custo de combinação é da ordem de $O(p)$, em que p representa o número de threads, e o código necessário é mínimo.

O contador privado com `critical` final e o vetor por thread com soma serial representam formas manuais do mesmo padrão, também com custo de combinação da ordem de $O(p)$.

2.3 Regiões críticas nomeadas e locks explícitos

`critical(nome)` utiliza um nome literal que é resolvido pelo compilador. Esse mecanismo é adequado para dois ou três recursos fixos, como `lista0` e `lista1`.

Entretanto, essa abordagem não escala para N recursos definidos em tempo de execução, pois não é possível indexar diretamente uma região crítica nomeada, como em:

```
1  /* Invalido: o nome da regioao critica nao pode ser uma variavel. */
2  #pragma omp critical(locks[i])
3      inserir(&heads[i], v);
```

Listing 1: Regiao critica nomeada nao indexavel

Para N dinâmico, utiliza-se um vetor de `omp_lock_t locks[N]`, com as operações `omp_init_lock`, `omp_set_lock`, `omp_unset_lock` e `omp_destroy_lock`.

Cada posição do vetor possui seu próprio lock, permitindo que recursos distintos sejam acessados simultaneamente.

3 Implementação

Foram utilizadas cinco versões em C11, todas com cláusula `default(none)` e medição de tempo com:

```
clock_gettime(CLOCK_MONOTONIC)
```

- v1: contador compartilhado com `critical` por hit;
- v2: contador compartilhado com `atomic` por hit;
- v3: contador privado com `critical` uma única vez;
- v4: vetor por thread com soma serial;
- v5: `reduction(+ hits)`.

A compilação foi realizada com as opções `-std=c11 -O2 -Wall -fopenmp -lm`, utilizando $N = 2.000.000$.

```
1 gcc -std=c11 -O2 -Wall -fopenmp v1_shared_critical_randr.c -o pi_v1 -lm
2
3 gcc -std=c11 -O2 -Wall -fopenmp v2_shared_atomic_randr.c -o pi_v2 -lm
4
5 gcc -std=c11 -O2 -Wall -fopenmp v3_private_critical_randr.c -o pi_v3 -lm
6
7 gcc -std=c11 -O2 -Wall -fopenmp v4_vector_randr.c -o pi_v4 -lm
8
9 gcc -std=c11 -O2 -Wall -fopenmp v5_reduction_randr.c -o pi_v5 -lm
10
11
12 ./execute.sh 2000000 3
```

Listing 2: Compilação e execução

4 Resultados e discussão

Foram avaliadas de 1 a 28 threads, com mediana de 3 repetições por ponto, em um Intel i7-14700HX sob WSL2.

O valor de `hits` permaneceu idêntico em todas as versões e para todas as quantidades de threads, demonstrando que a semente por *i* é reproduzível.

A versão v1 apresentou piora de desempenho com o aumento do número de threads. Como a sincronização ocorre a cada hit, o tempo passou de 0,02 s com 1 thread para 0,48 s com 28 threads.

A versão v2 permaneceu relativamente estável, entre aproximadamente 0,03 s e 0,04 s. A operação atômica apresenta custo inferior ao de `critical`, mas ainda ocorre sincronização a cada hit.

As versões v3, v4 e v5 apresentaram tempos entre 0,001 s e 0,002 s e escalaram bem até 14 núcleos. Esse comportamento era esperado e confirma que a privatização reduz significativamente o custo de sincronização.

A Figura 1 resume o comportamento em escala logarítmica, necessária porque v1 e v2 são ordens de grandeza mais lentos que v3–v5. A tabela numérica e o gráfico em escala linear foram suprimidos por redundância: repetiam os mesmos dados do gráfico log sem permitir distinguir v3, v4 e v5. Os valores completos permanecem disponíveis em `results.csv`.

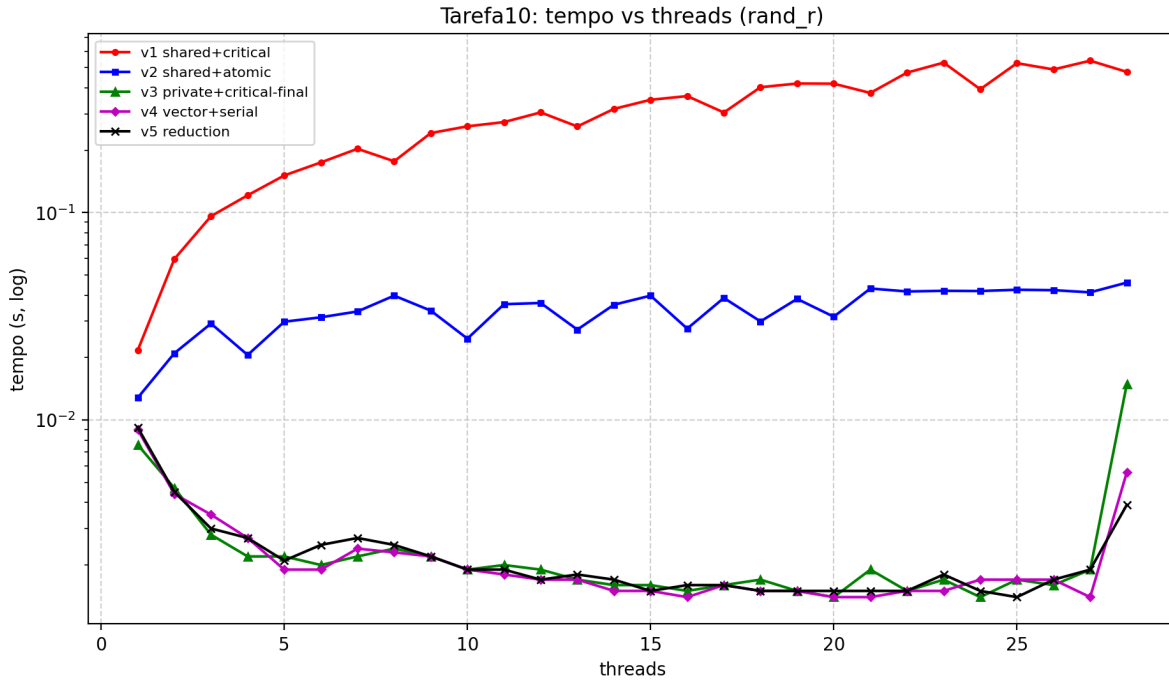


Figure 1: Tempo de execução em função do número de threads, em escala logarítmica, com $N = 2.000.000$ e mediana de 3 repetições.

5 Roteiro de uso

A escolha do mecanismo de sincronização depende principalmente do tipo de operação, da frequência de atualização e da quantidade de recursos compartilhados.

O roteiro de decisão apresentado na Figura 2 resume a escolha do mecanismo mais adequado.

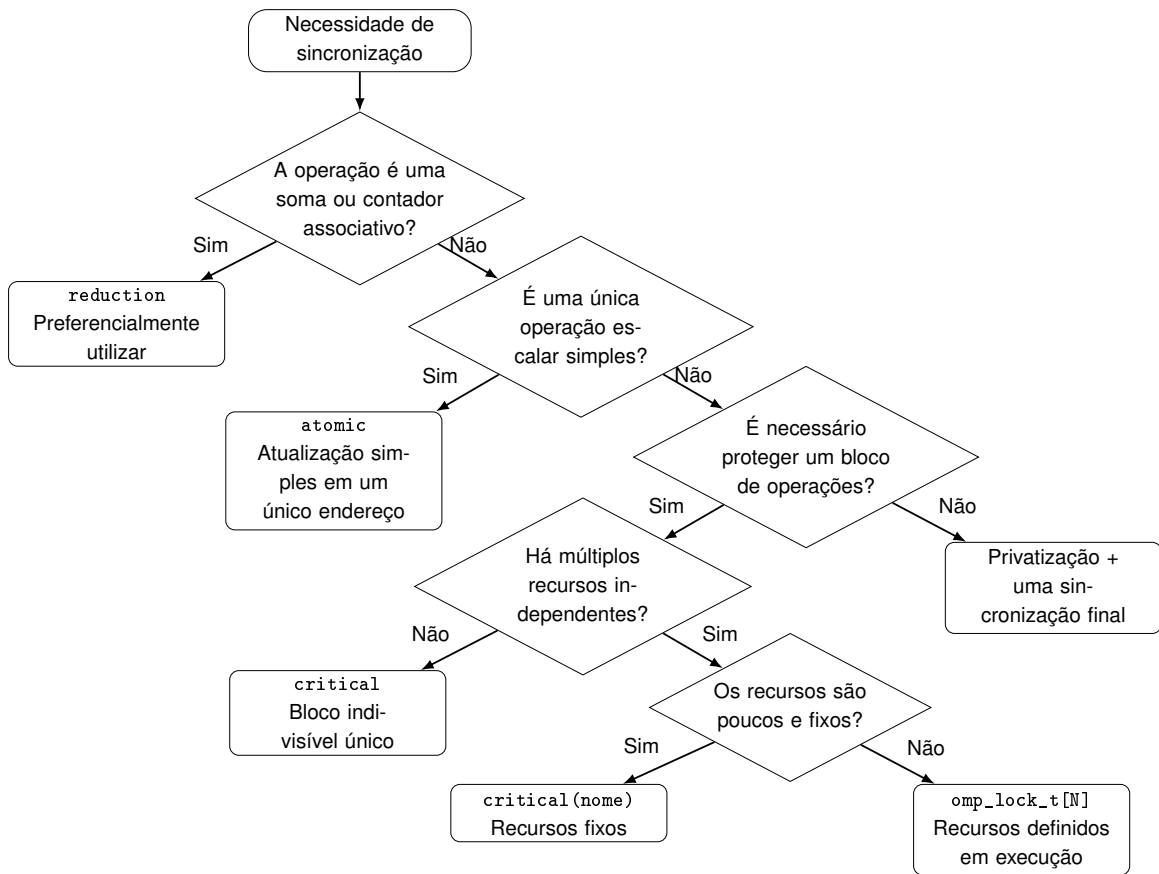


Figure 2: Diagrama de decisão para escolha do mecanismo de sincronização.

De forma resumida, os mecanismos podem ser utilizados conforme os seguintes critérios:

1. `reduction`: indicado para somas ou contadores associativos. É o caminho mais curto e, neste caso, apresenta o melhor desempenho.
2. Privado + 1 sincronização final: redução manual quando o valor combinado não é diretamente redutível, como no caso de múltiplas variáveis ou inserção em uma estrutura compartilhada.
3. `atomic` por update: indicado para atualizações frequentes em um único endereço e que possam ser expressas por uma operação atômica simples.
4. `critical` por update: indicado quando é necessário proteger um bloco contendo mais de uma operação que deve ser executada de forma indivisível.
5. `critical(nome)`: indicado quando existem dois ou três recursos fixos que precisam de mecanismos de exclusão mútua independentes.
6. `omp_lock_t[N]`: indicado quando existem N recursos e essa quantidade é definida em tempo de execução.

Como regra geral, recomenda-se realizar a sincronização por thread, com custo da ordem de $O(p)$, em vez de realizar a sincronização por hit, com custo da ordem de $O(N)$, salvo quando houver necessidade de visibilidade imediata.

6 Conclusão

Para o método de Monte Carlo utilizado, `reduction` apresentou o melhor equilíbrio entre desempenho e simplicidade. O mecanismo obtém tempo semelhante ao das versões v3 e v4 com uma implementação mais compacta, além de facilitar a leitura do código.

`atomic` apresentou desempenho superior ao de `critical` quando utilizado por hit, aproximadamente 10 vezes melhor com 28 threads no cenário avaliado, mas ainda apresentou desempenho inferior ao das abordagens baseadas em privatização.

A utilização de uma semente determinada pela iteração permitiu realizar a comparação mantendo os mesmos pontos gerados em todas as versões e quantidades de threads. Dessa forma, o efeito observado nos tempos está relacionado principalmente ao mecanismo de sincronização empregado.

Os valores numéricos completos encontram-se disponíveis em `results.csv`.

A Códigos-fonte das cinco versões

Os listados abaixo correspondem aos arquivos compilados com `-std=c11 -O2 -Wall -fopenmp -lm` e $N = 2.000.000$.

```
1 #define _POSIX_C_SOURCE 199309L
2
3 #include <omp.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <time.h>
7
8 static double agora(void) {
9     struct timespec t;
10    clock_gettime(CLOCK_MONOTONIC, &t);
11    return t.tv_sec + t.tv_nsec / 1e9;
12 }
13
14 int main(int argc, char *argv[]) {
15     long N = (argc > 1) ? atol(argv[1]) : 5000000L;
16     if (N <= 0) {
17         fprintf(stderr, "Uso: %s [Npontos]\n", argv[0]);
18         return 1;
19     }
20
21     long hits = 0;
22     double tempo = agora();
23
24     #pragma omp parallel shared(hits, N) default(none)
25     {
26
27         #pragma omp for
```

```

28     for (long i = 0; i < N; i++) {
29         unsigned int seed = 12345u + (unsigned int)(i * 2654435761u);
30         double x = (double)rand_r(&seed) / (double)RAND_MAX * 2.0 -
                1.0;
31         double y = (double)rand_r(&seed) / (double)RAND_MAX * 2.0 -
                1.0;
32         if (x * x + y * y <= 1.0) {
33             #pragma omp critical
34             hits++;
35         }
36     }
37 }
38
39 tempo = agora() - tempo;
40 printf("v1 shared+critical+rand_r: pi=%.6f tempo=%.4f s hits=%ld\n",
41        4.0 * hits / N, tempo, hits);
42 return 0;
43 }

```

Listing 3: v1: contador compartilhado com critical por hit. Versão base correta, porém lenta: cada hit serializa (O(N) entradas).

```

1  #define _POSIX_C_SOURCE 199309L
2
3  #include <omp.h>
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <time.h>
7
8  static double agora(void) {
9      struct timespec t;
10     clock_gettime(CLOCK_MONOTONIC, &t);
11     return t.tv_sec + t.tv_nsec / 1e9;
12 }
13
14 int main(int argc, char *argv[]) {
15     long N = (argc > 1) ? atol(argv[1]) : 5000000L;
16     if (N <= 0) {
17         fprintf(stderr, "Uso: %s [Npontos]\n", argv[0]);
18         return 1;
19     }
20
21     long hits = 0;
22     double tempo = agora();
23
24     #pragma omp parallel shared(hits, N) default(none)
25     {
26
27         #pragma omp for
28         for (long i = 0; i < N; i++) {

```

```

29         unsigned int seed = 12345u + (unsigned int)(i * 2654435761u);
30         double x = (double)rand_r(&seed) / (double)RAND_MAX * 2.0 -
                1.0;
31         double y = (double)rand_r(&seed) / (double)RAND_MAX * 2.0 -
                1.0;
32         if (x * x + y * y <= 1.0) {
33             #pragma omp atomic
34             hits++;
35         }
36     }
37 }
38
39 tempo = agora() - tempo;
40 printf("v2 shared+atomic+rand_r: pi=%.6f tempo=%.4f s hits=%ld\n",
41        4.0 * hits / N, tempo, hits);
42 return 0;
43 }

```

Listing 4: v2: contador compartilhado com atomic por hit. Mesma corretude de v1, menor sobrecarga com instrução atômica.

```

1  #define _POSIX_C_SOURCE 199309L
2
3  #include <omp.h>
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <time.h>
7
8  static double agora(void) {
9      struct timespec t;
10     clock_gettime(CLOCK_MONOTONIC, &t);
11     return t.tv_sec + t.tv_nsec / 1e9;
12 }
13
14 int main(int argc, char *argv[]) {
15     long N = (argc > 1) ? atol(argv[1]) : 5000000L;
16     if (N <= 0) {
17         fprintf(stderr, "Uso: %s [Npontos]\n", argv[0]);
18         return 1;
19     }
20
21     long total = 0;
22     double tempo = agora();
23
24     #pragma omp parallel shared(total, N) default(none)
25     {
26         long local = 0;
27
28         #pragma omp for
29         for (long i = 0; i < N; i++) {

```

```

30         unsigned int seed = 12345u + (unsigned int)(i * 2654435761u);
31         double x = (double)rand_r(&seed) / (double)RAND_MAX * 2.0 -
                1.0;
32         double y = (double)rand_r(&seed) / (double)RAND_MAX * 2.0 -
                1.0;
33         if (x * x + y * y <= 1.0) {
34             local++;
35         }
36     }
37
38     #pragma omp critical
39     total += local;
40 }
41
42 tempo = agora() - tempo;
43 printf("v3 private+critical-final+rand_r: pi=%.6f tempo=%.4f s hits=%
        ld\n",
44        4.0 * total / N, tempo, total);
45 return 0;
46 }

```

Listing 5: v3: contador privado com critical final. Custo $O(p)$: acumula sem lock, combina 1x por thread.

```

1  #define _POSIX_C_SOURCE 199309L
2
3  #include <omp.h>
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <time.h>
7
8  static double agora(void) {
9      struct timespec t;
10     clock_gettime(CLOCK_MONOTONIC, &t);
11     return t.tv_sec + t.tv_nsec / 1e9;
12 }
13
14 int main(int argc, char *argv[]) {
15     long N = (argc > 1) ? atol(argv[1]) : 5000000L;
16     if (N <= 0) {
17         fprintf(stderr, "Uso: %s [Npontos]\n", argv[0]);
18         return 1;
19     }
20
21     int P = omp_get_max_threads();
22     long *vec = calloc((size_t)P, sizeof(long));
23     if (vec == NULL) {
24         fprintf(stderr, "Falha ao alocar vetor de %d contadores\n", P);
25         return 1;
26     }

```

```

27
28     double tempo = agora();
29
30     #pragma omp parallel shared(vec, N) default(none)
31     {
32         int tid = omp_get_thread_num();
33         long local = 0;
34
35     #pragma omp for
36         for (long i = 0; i < N; i++) {
37             unsigned int seed = 12345u + (unsigned int)(i * 2654435761u);
38             double x = (double)rand_r(&seed) / (double)RAND_MAX * 2.0 -
39                     1.0;
40             double y = (double)rand_r(&seed) / (double)RAND_MAX * 2.0 -
41                     1.0;
42             if (x * x + y * y <= 1.0) local++;
43         }
44
45         vec[tid] = local;
46     }
47
48     long total = 0;
49     for (int t = 0; t < P; t++) total += vec[t];
50
51     tempo = agora() - tempo;
52     printf("v4 vector-per-thread+rand_r: pi=%.6f tempo=%.4f s hits=%ld\n",
53           4.0 * total / N, tempo, total);
54
55     free(vec);
56     return 0;
57 }

```

Listing 6: v4: vetor por thread com soma serial. Sem sincronização no laço; sem padding pode sofrer falso compartilhamento.

```

1  #define _POSIX_C_SOURCE 199309L
2
3  #include <omp.h>
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <time.h>
7
8  static double agora(void) {
9      struct timespec t;
10     clock_gettime(CLOCK_MONOTONIC, &t);
11     return t.tv_sec + t.tv_nsec / 1e9;
12 }
13
14 int main(int argc, char *argv[]) {
15     long N = (argc > 1) ? atol(argv[1]) : 5000000L;

```

```

16  if (N <= 0) {
17      fprintf(stderr, "Uso: %s [Npontos]\n", argv[0]);
18      return 1;
19  }
20
21  long hits = 0;
22  double tempo = agora();
23
24  #pragma omp parallel reduction(+:hits) default(none) shared(N)
25  {
26
27      #pragma omp for
28      for (long i = 0; i < N; i++) {
29          unsigned int seed = 12345u + (unsigned int)(i * 2654435761u);
30          double x = (double)rand_r(&seed) / (double)RAND_MAX * 2.0 -
31                  1.0;
32          double y = (double)rand_r(&seed) / (double)RAND_MAX * 2.0 -
33                  1.0;
34          if (x * x + y * y <= 1.0) hits++;
35      }
36
37      tempo = agora() - tempo;
38      printf("v5 reduction+rand_r: pi=%.6f tempo=%.4f s hits=%ld\n",
39            4.0 * hits / N, tempo, hits);
40      return 0;
41  }

```

Listing 7: v5: reduction. Versão idiomática: o runtime cria cópias privadas e combina no fim.